

DP - 01 - Design Pattern Creazionali

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory
- 5 Prototype
- 6 Singleton

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory
- 5 Prototype
- 6 Singleton

Definizione di Design Pattern

- I **Design Pattern** sono soluzioni tipiche a problemi ricorrenti nella progettazione software. Possono essere considerati come schemi predefiniti, personalizzabili per risolvere specifici problemi di design.
- A differenza di funzioni o librerie, un pattern non è un frammento di codice riutilizzabile, ma un concetto generale che guida la progettazione di una soluzione adattabile al contesto specifico.
- I pattern sono spesso confusi con gli algoritmi, poiché entrambi forniscono soluzioni a problemi noti. Tuttavia:
 - Un **algoritmo** definisce una sequenza di azioni ben precise per raggiungere un obiettivo.
 - Un **pattern** descrive una strategia generale di soluzione, il cui codice può variare a seconda del contesto applicativo.
- Un algoritmo può essere paragonato a una **ricetta di cucina**, con passaggi chiari e definiti. Un pattern, invece, è simile a un **progetto architettonico**, che fornisce una visione d'insieme lasciando libertà nell'implementazione.

Classificazione dei Design Pattern

- I pattern si suddividono in tre categorie principali:
 - **Creazionali**: gestiscono la creazione degli oggetti, migliorando flessibilità e riutilizzo del codice.
 - **Strutturali**: definiscono modalità di composizione di più oggetti e classi in strutture più complesse.
 - **Comportamentali**: regolano l'interazione e l'assegnazione delle responsabilità tra gli oggetti.

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory
- 5 Prototype
- 6 Singleton

Introduzione ai principi SOLID

- SOLID è un insieme di cinque principi per la progettazione software orientata agli oggetti:
 - Single responsibility principle o **principio di singola responsabilità**
 - Open/closed principle o **principio aperto/chiuso**
 - Liskov substitution principle o **principio di sostituzione di Liskov**
 - Interface segregation principle o **principio di segregazione delle interfacce**
 - Dependency inversion principle o **principio di inversione delle dipendenze**
- L'obiettivo è migliorare la manutenibilità, la scalabilità e la comprensibilità del codice.
- Ogni principio aiuta a evitare problemi comuni come codice rigido, difficile da modificare o estendere.

Single Responsibility Principle (SRP)

- Principio di singola responsabilità
- **Ogni classe dovrebbe essere responsabile di un'unica funzionalità.**
- Separare le responsabilità riduce la complessità e migliora la testabilità.

Esempio

Una classe che gestisce sia il salvataggio su database che la generazione di report viola SRP. È meglio suddividerla in due classi distinte.

Open/Closed Principle (OCP)

- Principio aperto/chiuso
- **Un'entità software dovrebbe essere aperta all'estensione, ma chiusa alla modifica.**
- È possibile aggiungere nuove funzionalità senza modificare il codice esistente.
- Questo si ottiene attraverso l'uso di astrazioni, ereditarietà e interfacce.

Esempio

Un sistema di pagamento dovrebbe poter supportare nuovi metodi senza modificare il codice esistente, utilizzando un'interfaccia comune.

Liskov Substitution Principle (LSP)

- Principio di sostituzione di Liskov
- Gli oggetti di una superclasse devono poter essere sostituiti con oggetti delle loro sottoclassi senza alterare la correttezza del programma.
- **Le sottoclassi devono poter sostituire le loro classi base senza alterare il comportamento del programma.**
- Un'istanza di una sottoclasse dovrebbe poter essere usata ovunque sia prevista una classe base.
- Violare LSP porta a errori imprevisti e codice difficile da mantenere.

Esempio

Se una sottoclasse ridefinisce un metodo in modo incompatibile con la classe base, il codice che usa la classe base potrebbe non funzionare correttamente.

Interface Segregation Principle (ISP)

- Principio di segregazione delle interfacce
- **Le interfacce devono essere specifiche per il client e non generiche.**
- È preferibile avere più interfacce piccole piuttosto che una sola interfaccia grande e generica.
- Una classe non dovrebbe essere obbligata a implementare metodi che non usa. È preferibile avere più interfacce specifiche piuttosto che un'unica interfaccia generale.
- I client non dovrebbero dipendere da metodi che non utilizzano.

Esempio

Un'interfaccia generica `IWorker` con metodi `lavorare()` e `farePausa()` obbligherebbe un robot a implementare un metodo inutile. È meglio separare le interfacce.

Dependency Inversion Principle (DIP)

- Principio di inversione delle dipendenze
- **I moduli di alto livello non devono dipendere da quelli di basso livello, entrambi devono dipendere da astrazioni.**
- Favorire le dipendenze da interfacce e astrazioni piuttosto che da implementazioni concrete.
- Aumenta la flessibilità e riduce l'accoppiamento tra componenti.

Esempio

Un modulo di gestione degli ordini non dovrebbe dipendere direttamente da una classe `Database`, ma da un'interfaccia `Storage`, in modo da poter utilizzare database diversi senza modifiche al codice.

Conclusione

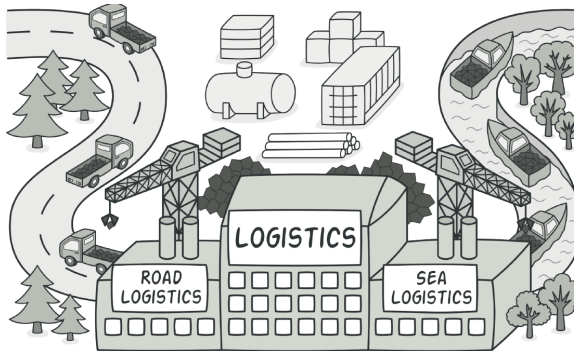
- I principi SOLID migliorano la qualità del software riducendo la complessità e aumentando la modularità.
- Seguire questi principi porta a codice più mantenibile, scalabile e testabile.
- L'adozione di SOLID deve essere bilanciata con le esigenze del progetto, evitando un'eccessiva astrazione.

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method**
- 4 Abstract Factory
- 5 Prototype
- 6 Singleton

Factory Method

- Il **Factory Method** fornisce un'interfaccia per la creazione di oggetti in una superclasse, ma consente alle sottoclassi di modificare il tipo di oggetti che verranno creati.
- Delegare la creazione di oggetti a una classe figlia invece di farlo direttamente.

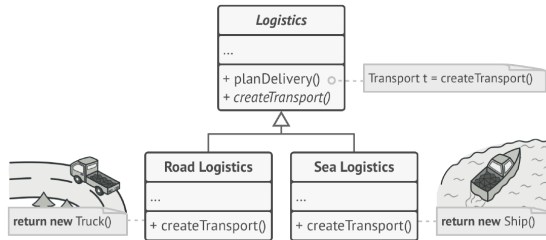


Problema

- Immaginate di dover sviluppare un'app per la gestione della logistica.
- La prima versione supporta solo il trasporto su camion, quindi tutto il codice principale finisce nella classe `Truck`.
- Dopo un po', l'app diventa popolare.
- Iniziate a ricevere richieste da aziende di trasporto marittimo che vogliono integrare le loro flotte nell'app.
- Il problema? Il codice dipende troppo dalla classe `Truck`.
- Aggiungere `Ship` significherebbe riscrivere gran parte del codice.
- E se poi doveste aggiungere anche treni o aerei?
- Il risultato finale sarebbe un codice caotico, pieno di `if` e `switch` che rendono tutto difficile da mantenere.

Soluzione (1)

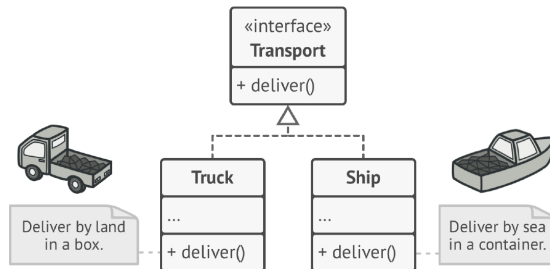
- Il pattern Factory Method suggerisce di sostituire le istanze dirette degli oggetti (`new`) con un *factory method* dedicato.
- Gli oggetti vengono comunque creati con `new`, ma dentro il *factory method*.
- Gli oggetti restituiti dal *factory method* vengono chiamati **prodotti**.



- A prima vista sembra solo uno spostamento del problema, ma c'è un vantaggio: possiamo ridefinire il *factory method* in una sottoclasse per cambiare il tipo di oggetto creato.

Soluzione (2)

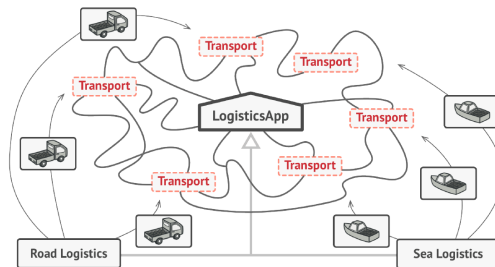
- C'è però un piccolo vincolo: le sottoclassi possono restituire solo prodotti che condividono una classe base o un'interfaccia comune.
- Per questo, il *factory method* nella classe base deve restituire un oggetto di tipo generico.



- Ad esempio, sia **Truck** che **Ship** dovrebbero implementare l'interfaccia **Transport**, che ha il metodo `deliver`.

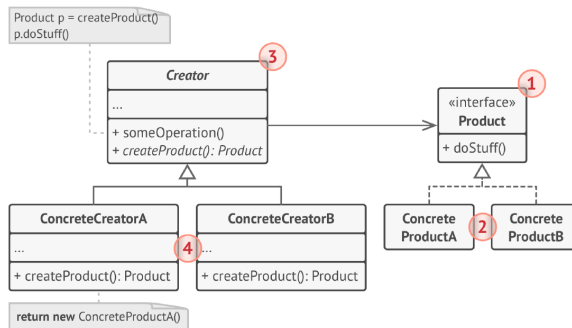
Soluzione (3)

- Ogni classe implementa `deliver` in modo diverso: i camion trasportano via terra, le navi via mare.
- Il *factory method* in `RoadLogistics` crea camion, mentre quello in `SeaLogistics` crea navi.
- Il codice che usa il *factory method* (detto **codice client**) non ha bisogno di sapere il tipo esatto di trasporto.
- Il client tratta tutti gli oggetti come `Transport` astratti.



- Il client sa solo che ogni oggetto `Transport` ha un metodo `deliver`, ma non si preoccupa di come funzioni nel dettaglio.

Struttura



- 1 Il **Product** definisce l'interfaccia comune a tutti i prodotti creati dal factory method.
- 2 I **Concrete Products** sono le implementazioni specifiche di questa interfaccia.
- 3 Il **Creator** dichiara il *factory method*, che restituisce nuovi oggetti.
- 4 I **Concrete Creators** sovrascrivono il *factory method* per restituire tipi specifici di prodotti.

Quando utilizzarlo

- Quando i tipi esatti e le dipendenze degli oggetti con cui il codice deve interagire non sono noti a priori.
 - Il Factory Method separa la logica di creazione degli oggetti dal codice che li utilizza, facilitando l'estensione della logica di costruzione senza modificare il resto del programma.
- Quando si desidera consentire agli utenti di una libreria di estendere i suoi componenti interni.
 - L'ereditarietà è un metodo semplice per estendere il comportamento predefinito di una libreria o framework. Tuttavia, è necessario un meccanismo per far sì che la libreria riconosca la nuova sottoclasse e la utilizzi al posto di un componente standard.
- Quando si vuole ottimizzare l'uso delle risorse di sistema riutilizzando oggetti esistenti anziché ricrearli ogni volta.
 - Questo approccio è utile per la gestione di oggetti complessi e ad alto consumo di risorse, come connessioni a database, file system e risorse di rete.

Come implementarlo

- 1 Fate in modo che tutti i prodotti seguano la stessa interfaccia.
- 2 Aggiungete un *factory method* nella classe principale.
- 3 Sostituite le istanze dirette dei prodotti con chiamate al *factory method*.
- 4 Create sottoclassi della classe principale per ogni tipo di prodotto e sovrascrivete il *factory method*.

Pro e Contro

Pro:

- Evita accoppiamenti stretti tra creatore e prodotti concreti.
- **Principio di Responsabilità Singola**: il codice di creazione è in un unico punto, quindi è più facile da mantenere.
- **Principio Open/Closed**: potete aggiungere nuovi prodotti senza modificare il codice esistente.

Contro:

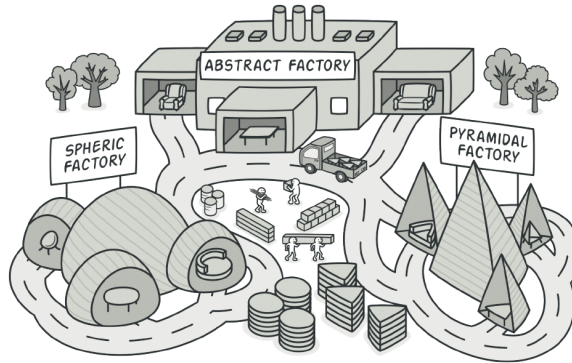
- Il codice può diventare più complesso, dato che servono più classi e sottoclassi.

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory**
- 5 Prototype
- 6 Singleton

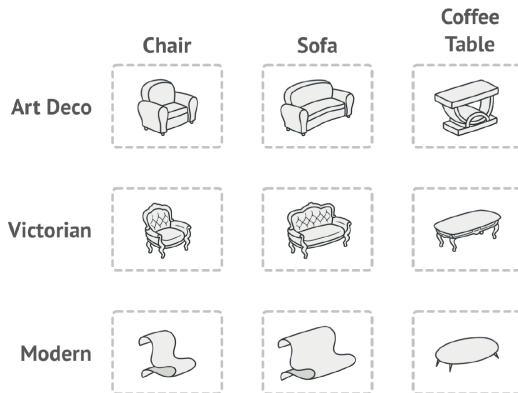
Abstract Factory

- **Abstract Factory** è un pattern creazionale che permette di produrre famiglie di oggetti correlati senza specificarne le classi concrete.



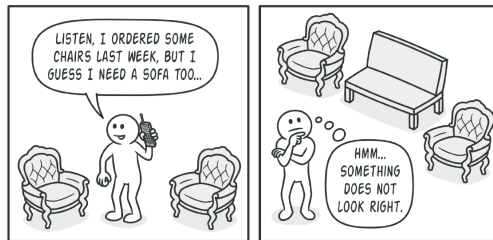
Problema (1)

- Immaginate di creare un simulatore di negozio di mobili. Il codice è composto da classi che rappresentano:
 - 1 Una famiglia di prodotti correlati, come `Chair`, `Sofa`, `CoffeeTable`.
 - 2 Varianti della famiglia di prodotti: `Modern`, `Victorian`, `ArtDeco`.



Problema (2)

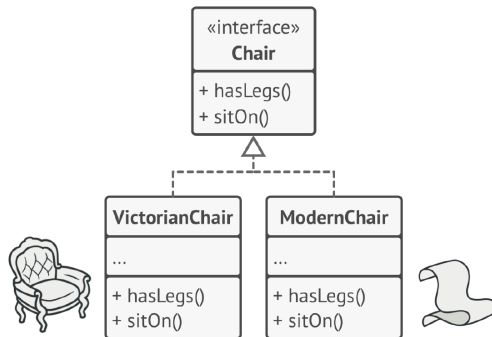
- Serve un modo per creare mobili che siano compatibili tra loro.
- I clienti si arrabbiano se ricevono mobili non coordinati.



- Inoltre, **non vogliamo** modificare il codice esistente per aggiungere nuovi prodotti o famiglie di prodotti.
- I cataloghi dei fornitori cambiano spesso, e il codice non dovrebbe essere modificato ogni volta.

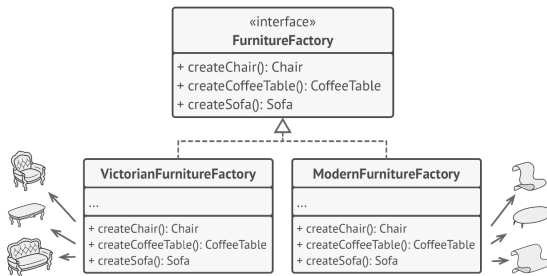
Soluzione (1)

- Il pattern **Abstract Factory** suggerisce di dichiarare esplicitamente interfacce per ogni prodotto della famiglia (es. `Chair`, `Sofa`, `CoffeeTable`).
- Tutte le varianti dei prodotti devono implementare queste interfacce.



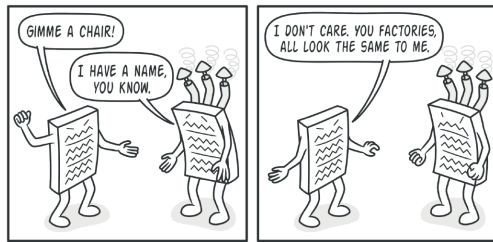
Soluzione (2)

- Dichiarare una *Abstract Factory*, un'interfaccia con metodi di creazione per tutti i prodotti della famiglia (`createChair`, `createSofa`, `createCoffeeTable`).
- Ogni variante della famiglia ha una sua factory concreta che implementa l'interfaccia della *AbstractFactory*.
- Es. *ModernFactory* crea solo *ModernChair*, *ModernSofa* e *ModernCoffeeTable*.



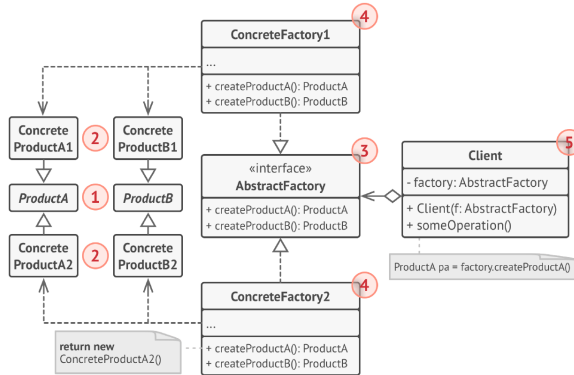
Soluzione (3)

- Il codice client lavora con factory e prodotti tramite interfacce astratte.
- Questo permette di cambiare factory e prodotti senza modificare il codice client.



- Il client richiede una sedia alla factory senza conoscere la classe concreta della factory o del prodotto.
- Il client usa solo l'interfaccia astratta `Chair`, sapendo solo che essa implementa il metodo `sitOn()`.
- Tutti i prodotti creati dalla stessa factory sono compatibili tra loro.
- La creazione della factory concreta avviene all'inizio dell'applicazione in base alla configurazione o all'ambiente.

Struttura



- 1 **Prodotti Astratti:** dichiarano interfacce per prodotti correlati.
- 2 **Prodotti Concreti:** implementazioni specifiche delle interfacce, raggruppate per variante.
- 3 **Abstract Factory:** dichiara metodi per creare i prodotti astratti.
- 4 **Factory Concrete:** implementano l'Abstract Factory, creando solo prodotti di una variante specifica.
- 5 **Client:** usa le factory tramite interfacce astratte, senza dipendere dalle implementazioni concrete.

Quando usarlo

- Quando il codice deve gestire diverse famiglie di prodotti correlati senza dipendere dalle loro classi concrete, permettendo estensibilità futura.
 - L'**Abstract Factory** fornisce un'interfaccia per la creazione di oggetti appartenenti alla stessa famiglia di prodotti.
 - Utilizzando questa interfaccia, si evita il rischio di creare varianti incompatibili con i prodotti già utilizzati dall'applicazione.

Come implementarlo

- 1 Definire una matrice di tipi di prodotti e varianti.
- 2 Dichiarare interfacce astratte per tutti i prodotti.
- 3 Dichiarare un'interfaccia Abstract Factory con metodi di creazione.
- 4 Implementare una factory concreta per ogni variante di prodotto.
- 5 Creare la factory concreta all'avvio dell'applicazione.
- 6 Sostituire tutte le istanze dirette di prodotti con chiamate ai metodi di factory.

Pro e Contro

Pro:

- Assicura la compatibilità tra i prodotti creati.
- Riduce il legame stretto tra client e classi concrete.
- Segue il principio di responsabilità unica.
- Segue il principio Open/Closed: nuove varianti senza modificare il codice esistente.

Contro:

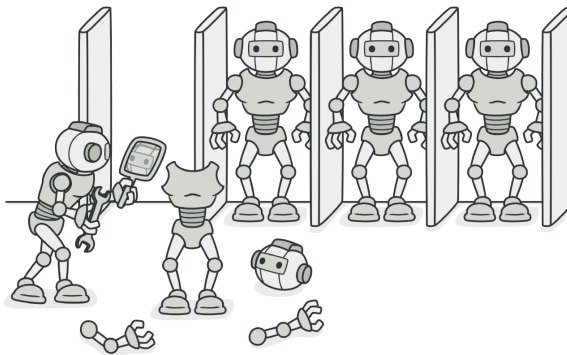
- Aggiunge complessità introducendo molte interfacce e classi.

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory
- 5 Prototype**
- 6 Singleton

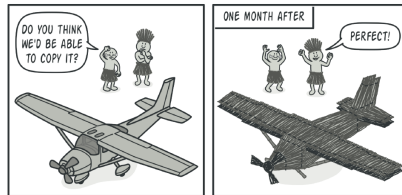
Prototype

- Il **Prototype** è un design pattern creazionale che consente di copiare oggetti esistenti senza rendere il codice dipendente dalle loro classi.
- Conosciuto anche come: **Clone**



Problema

- Supponiamo di avere un oggetto e di voler creare una sua copia esatta.
- Come fare? Prima di tutto, bisogna creare un nuovo oggetto della stessa classe.
- Poi bisogna copiare tutti i valori dei campi dell'oggetto originale nel nuovo oggetto.
- Sembra semplice! Ma c'è un problema: non tutti gli oggetti possono essere copiati in questo modo, poiché alcuni campi potrebbero essere privati e quindi non accessibili dall'esterno.
- C'è anche un altro problema: dovendo conoscere la classe dell'oggetto per crearne un duplicato, il codice diventa dipendente da quella classe.
- Se questa dipendenza non è un problema, c'è un'ulteriore complicazione: a volte si conosce solo l'interfaccia dell'oggetto e non la sua classe concreta, come accade quando un metodo accetta qualsiasi oggetto che segue una determinata interfaccia.



Soluzione (1)

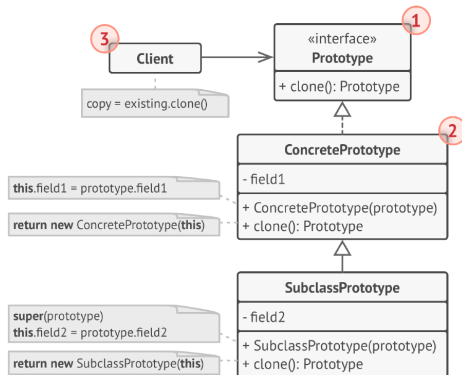
- Il pattern **Prototype** delega il processo di clonazione agli stessi oggetti che devono essere clonati.
- Il pattern dichiara un'interfaccia comune per tutti gli oggetti che supportano la clonazione.
- Questa interfaccia permette di clonare un oggetto senza accoppiare il codice alla classe concreta dell'oggetto stesso.
- Di solito, questa interfaccia contiene un unico metodo di clonazione.
- L'implementazione del metodo di clonazione è simile in tutte le classi.
- Il metodo crea un nuovo oggetto della stessa classe e copia tutti i valori dei campi dal vecchio oggetto al nuovo.
- È possibile copiare anche i campi privati, poiché la maggior parte dei linguaggi di programmazione consente agli oggetti di accedere ai campi privati di altri oggetti della stessa classe.

Soluzione (2)

- Un oggetto che supporta la clonazione viene chiamato **Prototype**.
- Quando un oggetto ha decine di campi e centinaia di possibili configurazioni, la clonazione può essere un'alternativa alla sottoclassificazione.
- Il funzionamento è semplice: si creano alcuni oggetti preconfigurati.
- Quando serve un oggetto simile a uno di quelli preconfigurati, si clona il prototipo invece di costruire un nuovo oggetto da zero.



Struttura



- 1 L'interfaccia **Prototype** dichiara i metodi di clonazione, che consistono in un unico metodo `clone()`.
- 2 La classe **Concrete Prototype** implementa la clonazione. Oltre a copiare i dati originali, può gestire casi particolari come la clonazione di oggetti collegati o la risoluzione di dipendenze ricorsive.
- 3 Il **Client** può generare una copia di qualsiasi oggetto che segue l'interfaccia del prototipo.

Quando utilizzarlo

- Quando il codice non deve dipendere dalle classi concrete degli oggetti da copiare.
 - Questo è frequente quando si lavora con oggetti ricevuti da codice di terze parti tramite un'interfaccia, rendendo sconosciute le loro classi concrete.
- Quando si desidera ridurre il numero di sottoclassi che differiscono solo per la configurazione degli oggetti.
 - Il **Prototype** consente di utilizzare un insieme di oggetti preconfigurati come prototipi.
 - Invece di istanziare una sottoclasse con una configurazione specifica, il client può selezionare un prototipo adeguato e clonarlo.

Come implementarlo

- 1 Creare un'interfaccia **Prototype** e dichiarare il metodo `clone()`.
- 2 Le classi concrete devono implementare il metodo `clone()` copiando tutti i campi.
- 3 Il metodo di clonazione può consistere in una sola riga di codice: una chiamata a un costruttore di copia.
- 4 Opzionalmente, si può creare un **PrototypeRegistry** per gestire gli oggetti clonabili più usati.
- 5 Sostituire le chiamate dirette ai costruttori delle sottoclassi con richieste al registro dei prototipi.

Pro e Contro

Pro:

- Permette di clonare oggetti senza accoppiare il codice alle loro classi concrete.
- Riduce la duplicazione del codice di inizializzazione.
- Permette di creare oggetti complessi in modo più efficiente.
- È un'alternativa all'ereditarietà per la creazione di configurazioni predefinite.

Contro:

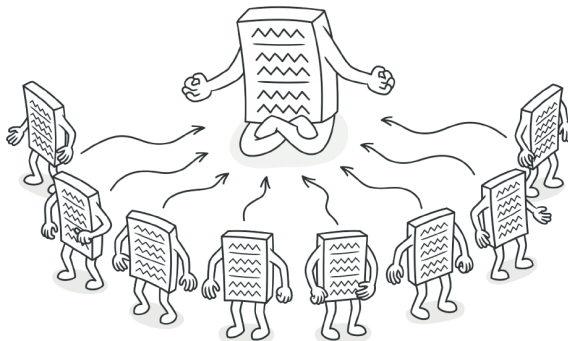
- La clonazione di oggetti complessi con riferimenti circolari può essere difficile da gestire.

Argomenti trattati

- 1 Introduzione
- 2 Principi SOLID
- 3 Factory Method
- 4 Abstract Factory
- 5 Prototype
- 6 Singleton

Singleton

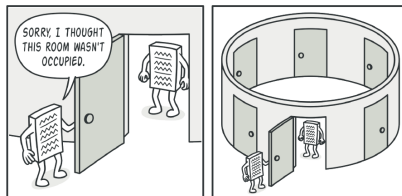
- **Singleton** è un pattern creazionale che garantisce l'esistenza di un'unica istanza di una classe, fornendo un punto di accesso globale ad essa.



Problema

Il pattern Singleton affronta **due problematiche**, violando il Principio di singola responsabilità:

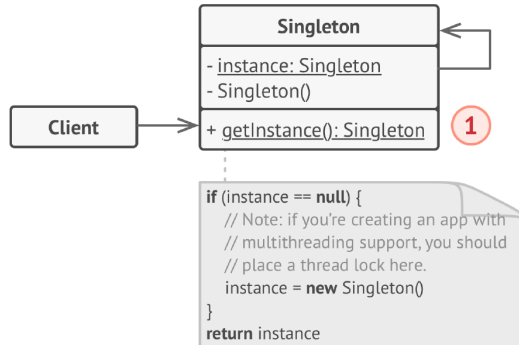
- Garantire un'unica istanza della classe. Questo è utile per controllare l'accesso a risorse condivise, come database o file.
 - Se si tenta di creare una nuova istanza, viene restituita quella già esistente.
 - Un costruttore tradizionale non può implementare questo, poiché restituisce sempre un nuovo oggetto.
- Fornire un punto di accesso globale all'istanza, evitando i rischi delle variabili globali, che possono essere sovrascritte accidentalmente.
 - Il Singleton consente l'accesso centralizzato all'istanza senza comprometterne l'integrità.



Soluzione

- Rendere il costruttore privato per impedire la creazione diretta di istanze.
- Definire un metodo statico che crea e memorizza l'istanza alla prima chiamata, restituendola nelle successive.
- Il Singleton è accessibile a qualsiasi parte del programma tramite il metodo statico, che garantisce il riutilizzo della stessa istanza.

Struttura



- La classe Singleton definisce un metodo statico che restituisce sempre la stessa istanza.
 - Il costruttore è nascosto al codice client.
 - L'unico modo per ottenere l'istanza è chiamare il metodo `getInstance`.

- Quando una classe deve avere un'unica istanza accessibile globalmente, ad esempio per la gestione di un database condiviso.
 - Il Singleton impedisce la creazione multipla di oggetti, restituendo sempre l'istanza esistente.
- Quando è necessario un controllo rigoroso sulle variabili globali.
 - Il Singleton garantisce l'unicità dell'istanza, impedendone la sostituzione da parte di codice esterno.

Implementazione

- 1 Dichiarare un campo statico privato per memorizzare l'istanza.
- 2 Definire un metodo statico pubblico per l'accesso all'istanza.
- 3 Implementare l'"inizializzazione pigra": l'istanza viene creata alla prima richiesta e riutilizzata successivamente.
- 4 Rendere il costruttore privato per impedire la creazione diretta di nuove istanze.
- 5 Modificare il codice client sostituendo le istanziazioni dirette con chiamate al metodo statico.

Pro e Contro

Vantaggi:

- Garantisce l'unicità dell'istanza della classe.
- Fornisce un punto di accesso globale all'istanza.
- L'istanza viene creata solo alla prima richiesta, ottimizzando l'uso delle risorse.

Svantaggi:

- Viola il Principio di singola responsabilità, combinando gestione dell'istanza e accesso globale.